

**COST-AWARE MULTI-AGENT
SEARCH AND RESCUE: A
REINFORCE BASELINE AND A
MAPPO UPGRADE WITH
COMMUNICATION**

CONTENTS:

- 01. Introduction
- 02. Problem Statement
- 03. Proposed Solution
- 04. Literature Review
- 05. Methodology Overview
- 06. Environment and Data Setup
- 07. Agent and Algorithm Design
- 08. Training Paradigm
- 09. Simulation
- 10. Implementation Flow
- 11. Experimental Results
- 12. Conclusion

INTRODUCTION

- Simulated multi drone search and rescue (SAR) task using MARL.
- Two versions built in sequence: v1 (REINFORCE baseline) and v2 (MAPPO with communication).
- v1 exposes a critical training bug; v2 fixes it and extends the task.
- Theme: cost aware agents must weigh every action, including talking to teammates, against a battery budget.

PROBLEM STATEMENT

- Coordinate 7 drones to find 7 survivors on a grid as fast as possible.
- Shared constraint: limited battery per drone.
- Partial observability: a drone only knows a survivor's location once it physically visits that cell.
- Coordination is only via fixed sector assignments, not learned.
- Secondary problem: getting the RL training loop itself correct (v1 fails at this).



PROPOSED SOLUTION

Stage 1 (v1): independent REINFORCE policy per drone, no communication, reward shaping for search efficiency.

Stage 2 (v2):

- Fix v1's core bug (policy now persists across episodes).
- Upgrade to MAPPO: decentralized actors + one centralized critic (CTDE).
- Add a 6th action, COMMUNICATE, with battery cost and reward shaping.
- Both README files double as diagnostic reports, not just result summaries.

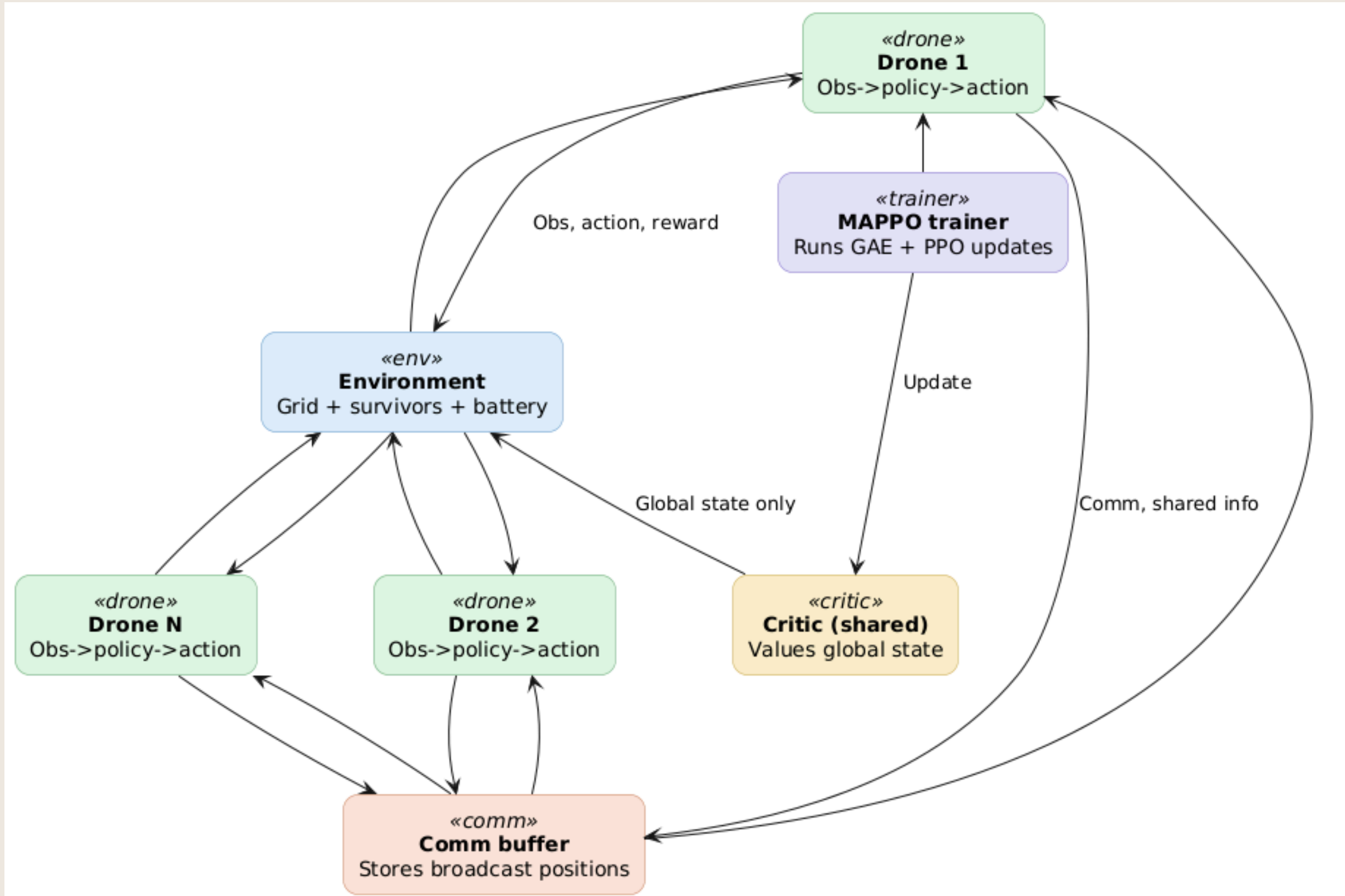
LITERATURE REVIEW

S. No.	Focused Area	Methodology	Key Findings	Relevance
1.	Distributed UAV Search and Rescue under communication constraints	Cooperative cluster-based coordination with restricted inter-UAV communication	Achieved effective SAR coverage despite limited communication by cooperative clustering	Closest application-level ancestor addressing SAR missions under communication constraints
2.	Communication-aware cooperative multi-UAV target tracking	Integrated communication quality into state representation and reward design	Improved tracking accuracy by explicitly accounting for communication degradation	Directly supports communication-aware state and reward modeling in multi-UAV systems
3.	Decentralized UAV formation under uncertainty	Formal Dec-POMDP-based decentralized planning framework	Provided theoretical guarantees for decentralized coordination under partial observability	Strengthens justification for belief-level coordination under partial observability
4.	Belief-based decentralized coordination with minimal communication	POMDP-based coordination combined with consensus mechanisms	Enabled robust coordination with limited inter-agent communication	Strengthens justification for belief-level coordination under partial observability
5.	Communication-aware UAV swarm formation control	Deep Q-Network-based control incorporating communication feedback	Improved formation stability under communication degradation	Bridges communication-aware theory with learning-based control policy design

METHODOLOGY

- Build shared environment (grid, drones, survivors, battery, rewards)
- Implement v1, observe flat results, diagnose root cause
- Root cause: policy network rebuilt every episode instead of persisting
- Rebuild as v2: fix bug, upgrade algorithm, add communication and partial observability
- Train longer (150 vs 40 episodes), track many more metrics
- Document what still doesn't work instead of declaring it finished

METHODOLOGY



ENVIRONMENT AND DATA SETUP

- **No external dataset;** environment is procedurally generated each episode.
- **Common:** 11x11 grid, 7 drones from shared base, 7 survivors placed randomly each episode, fixed sector zones.
- **v1:** battery 20, 5 actions, 40 steps per episode, 40 episodes, 4 dim state (x, y, battery, survivors left).
- **v2:** battery 100, 6 actions, 80 steps per episode, 150 episodes.
- **11 dim actor observation:** base 4 dims plus up to 3 shared survivor positions (2 each) plus a comm flag.
- **Separate 37 dim global state (critic only):** full survivor and drone ground truth.

AGENT AND ALGORITHM DESIGN

- **v1:** PolicyNet is a single Linear(4,5) plus softmax, no hidden layer, one Adam optimizer per drone, plain REINFORCE update.

- **v2:**
Actor per drone: 11 to 64 to 64 to 6, ReLU, softmax,
orthogonal init
One shared CentralizedCritic: 37 to 128 to 128 to 1,
training only

COMMUNICATE action (index 5) triggers no movement,
costs battery, writes to shared buffer

Buffer tracks broadcasts, detects new info, lets other
drones read up to 3 shared positions.

TRAINING PARADIGM

v1: full episode rollout, then one REINFORCE step per drone

- Core bug: new GridWorld and PolicyNet created every episode inside the loop, so weights never carry over.
- Secondary bug: $LR = 0.01$ in config never passed to optimizer, so it silently trains at default 0.001.

v2: CTDE PPO

- Rollout full episode, compute GAE advantages and returns via critic.
- 4 epochs, batch size 64, clipped surrogate plus entropy bonus plus value loss plus grad clipping.
- Actor LR $3e-4$, critic LR $1e-3$, separate optimizer per drone actor.
- Fix: drones and actors built once, reset (not rebuilt) every episode, so learning persists.
- Known issue: leftover debug print statements fire on every update.

IMPLEMENTATION FLOW

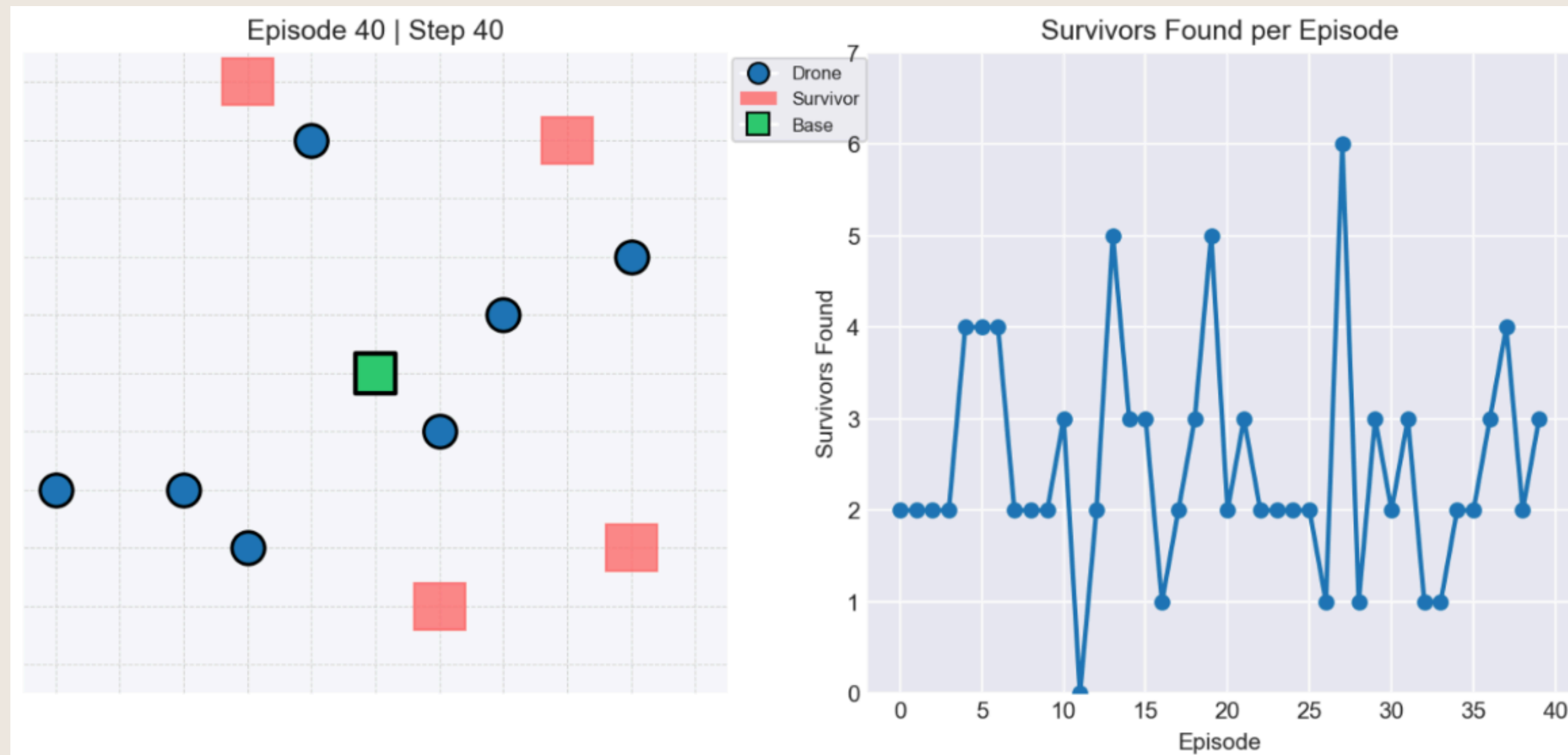
- v1: config sets constants, then main.py builds a new GridWorld each episode, which spawns drones with fresh policies (the bug), runs action selection and movement with reward shaping, applies a REINFORCE update at episode end, then visualizes and saves.
- v2: config sets constants, main.py builds one persistent GridWorld and one MAPPOTrainer, each episode resets (not rebuilds) drones and clears the comm buffer, the step loop gets obs (with shared info), selects actions, env.step handles movement, communication, and rewards, transitions are stored, then the trainer runs GAE plus PPO update per drone, metrics are logged, and after 150 episodes a summary figure is saved.

SIMULATION

- v1: live 2 panel matplotlib view (grid plus survivors found curve), updates every 3 steps, saved to PNG at the end.
- v2: live grid panel plus 5 rolling metric panels (reward, accuracy, losses, entropy, comm usage), updated continuously.
- v2 has a headless fallback: tries multiple GUI backends, falls back to save only mode if none work.
- Both simulations confirmed functional by checking the actual saved result images.

EXPERIMENTAL RESULTS

v1: survivors found bounces between 1 and 4 most episodes, occasional spikes to 5 to 6, never 7 of 7, no upward trend, matching expectation given the persistence bug



v2:

Reward rolling average climbs from about -190 to about -60/-70 across training

Rescue accuracy usually 60 to 90% of survivors found per episode

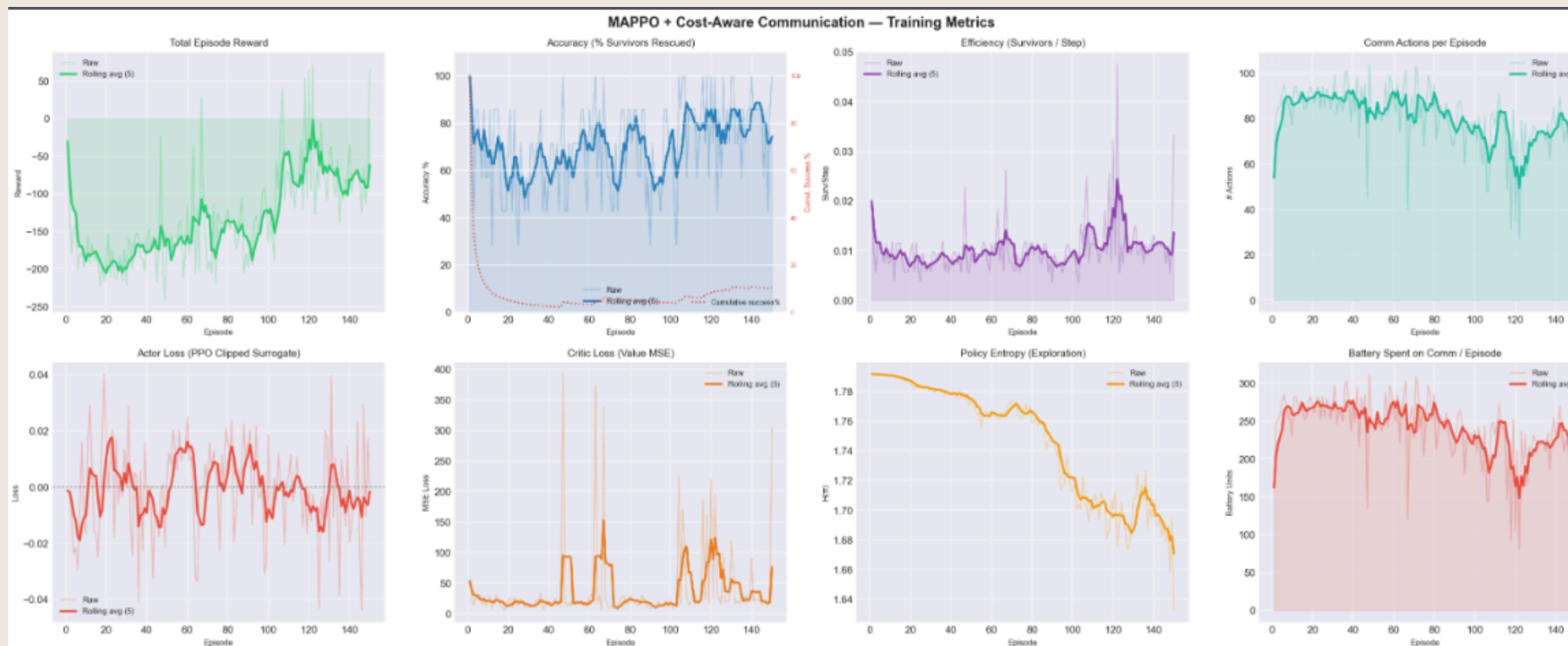
Full team clears (7 of 7) remain rare, cumulative success rate stays low overall

Comm usage settles at 70 to 90 actions per episode within the first 20 episodes and stays flat, with no sign of learning to communicate more selectively

Entropy drifts down from about 1.79 to 1.63, policy gets more decisive without collapsing

Critic loss spikes hard around episodes 45, 60 to 65, and 100 to 120 (up to 300 to 400), unresolved

Best episode: final episode, 7 of 7 rescued in 30 steps using 21 comms



CONCLUSION:

- v1 shows a broken but plausible looking baseline: correct environment logic undone by a training loop bug
- v2 shows real learning once the bug is fixed and MAPPO plus communication are added: reward climbs, rescue rate improves substantially
- Full board clears are still rare, suggesting the last 1 to 2 survivors need dedicated reward shaping
- Communication converges to "communicate a lot" rather than "communicate smart," suggesting current comm cost shaping needs sharpening
- Critic loss instability spikes remain unexplained and unresolved
- Minor cleanup items remain: dead config line, leftover debug prints
- Overall: the task is workably solved with visible learning, but explicitly not fully solved, and the project is transparent about that gap



THANK
YOU